```
localize  # Localize Date and Time objects to local formats
```

These have the aliases #t and #l so you can use them like this:

```
I18n.t 'store.title'
I18n.l Time.now
```

There are also attribute readers and writers for the following attributes:

```
load_path         # Announce your custom translation files
locale            # Get and set the current locale
default_locale    # Get and set the default locale
exception_handler # Use a different exception_handler
backend           # Use a different backend
```

So, let's internationalize a simple Rails application from the ground up in the next chapters!

2 Setup the Rails Application for Internationalization

There are just a few simple steps to get up and running with I18n support for your application.

2.1 Configure the I18n Module

Following the *convention over configuration* philosophy, Rails will set up your application with reasonable defaults. If you need different settings, you can overwrite them easily.

Rails adds all `.rb` and `.yml` files from the `config/locales` directory to your **translations load path**, automatically.

The default `en.yml` locale in this directory contains a sample pair of translation strings:

```
en:
  hello: "Hello world"
```

This means, that in the `:en` locale, the key *hello* will map to the *Hello world* string. Every string inside Rails is internationalized in this way, see for instance Active Model validation messages in the `activemodel/lib/active_model/locale/en.yml` file or time and date formats in the `activesupport/lib/active_support/locale/en.yml` file. You can use YAML or standard Ruby Hashes to store translations in the default (Simple) backend.

The I18n library will use **English** as a **default locale**, i.e. if you don't set a different locale, `:en` will be used for looking up translations.

The i18n library takes a **pragmatic approach** to locale keys (after some discussion), including only the *locale* ("language") part, like `:en`, `:pl`, not the *region* part, like `:en-US` or `:en-GB`, which are traditionally used for separating "languages" and "regional setting" or "dialects". Many international applications use only the "language" element of a locale such as `:cs`, `:th` or `:es` (for Czech, Thai and Spanish). However, there are also regional differences within different language groups that may be important. For instance, in the `:en-US` locale you would have $ as a currency symbol, while in `:en-GB`, you would have £. Nothing stops you from separating regional and other settings in this way: you just have to provide full "English - United Kingdom" locale in a `:en-GB` dictionary. Few gems such asGlobalize3 may help you implement it.

The **translations load path** (`I18n.load_path`) is just a Ruby Array of paths to your translation files that will be loaded automatically and available in your application. You can pick whatever directory and translation file naming scheme makes sense for you.

The backend will lazy-load these translations when a translation is looked up for the first time. This makes it possible to just swap the backend with something else even after translations have already been announced.